

Assignment 1: Build, Observe, and Explain Linux Processes

Linux Systems and Performance - Week 1

Process Execution · System Calls · Process Observation · CPU Monitoring

1. Assignment Philosophy

This assignment is not evaluated only by whether the code runs. You will build Linux process-management tools, observe their real execution with Linux observability tools, validate your results against the system, and explain the behavior using the Week 1 execution model.

BUILD -> OBSERVE -> ANALYZE -> EXPLAIN

Important: All coding tools may be used. However, you are responsible for understanding, debugging, modifying, and explaining all submitted code and experimental results. Implementation alone is not sufficient for full credit.

Level	Build	Observe / Validate	Main Concept
Level 1	Tiny Shell	strace -f	fork -> exec -> wait, FD inheritance, pipe
Level 2	Custom ps System Call	Compare with Linux ps	task_struct, PID/PPID/TGID/state, copy_to_user
Level 3	Custom CPU Monitor (mytop)	Compare with top/ps/perf	snapshot delta, CPU usage, process state

2. Common Rules

- Work on the Linux/kernel environment specified by the instructor.
- Use C for the required implementation unless otherwise approved.
- Do not use system() to implement shell command execution.
- For Levels 2-3, do not read /proc or execute the system ps/top commands to generate your program's required output.
- System ps/top/perf may be used only for validation and observation.
- Record the exact commands used for experiments. Screenshots alone are not sufficient; include short interpretations.
- Kernel-version-dependent details must be documented when they affect your implementation.

3. Level 1 - Tiny Shell + Execution Trace

3.1 Required Implementation

- Display a prompt and parse a command line into program name and arguments.
- Run an external command using fork() -> execvp() -> waitpid().
- Implement built-in cd, pwd, and exit in the shell process. Implement pwd using getcwd() (or an equivalent direct API), not by executing the external pwd program.
- Implement at least one two-command pipeline using pipe() and dup2(), for example: ls | wc -l.
- Close unused pipe/file descriptors correctly and perform basic error handling.
- Background jobs, SIGCHLD handling, and Ctrl-C forwarding are intentionally deferred to Assignment 2.

```
myshell> /bin/echo hello
hello
```

```
myshell> pwd
/home/student
myshell> cd /tmp
myshell> pwd
/tmp
```

```
myshell> ls | wc -l
23      # example output; the actual count depends on the directory
```

3.2 Required Observation with strace

Run your shell under strace and preserve child-process traces.

```
strace -f -o shell.trace ./myshell
```

Inside myshell, execute at least:

```
myshell> /bin/echo hello
myshell> ls | wc -l
myshell> cd /tmp
myshell> pwd
myshell> exit
```

From shell.trace, identify evidence of the following operations:

Operation	Evidence to Find	What It Demonstrates
Process creation	fork/clone-family trace entry	new execution context is created
Program replacement	execve(...)	child replaces its program image
Parent synchronization	wait4/wait-family trace entry	parent waits/reaps child
Pipeline creation	pipe/pipe2	kernel pipe FDs are created
FD redirection	dup2/dup3	stdin/stdout is redirected to pipe ends
FD cleanup	close	unused descriptors are released
Working directory built-ins	chdir("/tmp"), getcwd(...)	cd changes and pwd queries the shell process current working directory

3.3 Required Explanation

1. For one external command, identify which process executes execve(). Does exec change that process's PID?
2. Why must cd be implemented as a shell built-in rather than by forking a child that calls chdir()?
3. Why is pwd useful as a shell built-in in this assignment, and which system call/library interface does your implementation use to obtain the shell's current working directory?
4. For your pipeline, draw or describe which process owns each pipe end after fork().
5. Why is dup2() needed in the pipeline?
6. What can happen if the parent or child keeps an unnecessary pipe write end open?
7. Relate at least four lines from your strace output to the sequence fork -> exec -> wait.

4. Level 2 - Custom ps System Call + Validation

4.1 Goal

Add a custom Linux system call that obtains a snapshot of process information in the kernel and safely returns it to a user-space program. The required view is process-oriented; per-thread listing is optional.

4.2 Required Output Fields

Field	Meaning
PID	process/thread ID selected for the process-oriented snapshot
PPID	parent process ID
TGID	thread-group ID
STATE	documented human-readable or compact state mapping
COMMAND	task command name (comm)

```
$ ./mysps
PID  PPID  TGID  STATE  COMMAND
3201  2900  3201  S      sleep
3202  2900  3202  R      cpuwork
...
```

4.3 Kernel-Side Requirements

- Define and register the custom system call using the instructor-provided kernel source/build environment.
- Collect a bounded snapshot; do not return an unbounded amount of kernel data.
- Do not directly dereference/write arbitrary user-space pointers from the kernel.
- Use an appropriate safe kernel-to-user copy mechanism such as copy_to_user().
- Document whether your implementation displays only process leaders or individual threads. Explain your selection criterion (for example, the PID/TGID relationship), and describe how kernel task states are mapped to the STATE values shown by your program.
- Check return values and handle invalid user arguments.

Important: The exact syscall registration files and some task/state details are architecture- and kernel-version-dependent. Use the course kernel as the authority and document the files you changed.

4.4 Required Validation Experiment

Create two visibly different processes and capture both your custom snapshot and the standard Linux view close together in time.

```
sleep 30 &
# Use the instructor-provided cpuwork program exactly as specified for this experiment.
./cpuwork --threads 1 &

./mysps
ps -eo pid,ppid,stat,comm
```

Include a short comparison table for selected processes:

Process	mysps PID/PPID/State	Linux ps PID/PPID/STAT	Match? / Explanation
sleep			
cpuwork			
myshell or another selected process			

4.5 Required Explanation

- Where does the kernel obtain PID, PPID, TGID, state, and command information?
- Why is `copy_to_user()` used instead of directly writing through a user pointer?
- Why is a sleeping process commonly observed differently from a CPU-bound runnable/running process?
- When are PID and TGID the same, and when can they differ?
- Why can mysps and Linux ps disagree for a process even when both implementations are correct?
- Identify the user-to-kernel path of your new system call: syscall number/name registration -> architecture dispatch -> handler -> copy back to user.

5. Level 3 - Custom CPU Monitor (mytop)

5.1 Goal

Extend process observation into a periodic CPU monitor. Your program must estimate CPU usage from two snapshots separated in time, rank processes, and display the most CPU-active processes.

5.2 Required Behavior

- Display PID, CPU%, STATE, and COMMAND.
- Refresh every 2 seconds.
- Compute CPU usage from the change in process CPU time between snapshots divided by elapsed wall time, using one clearly documented normalization convention.
- Sort by CPU usage and display the Top 10 processes.
- Handle processes that appear or disappear between snapshots reasonably.
- Ctrl-C should terminate the monitor cleanly.
- Do not use `/proc` or invoke `top/ps` to generate the required mytop values.

```
$ ./mytop
```

```
PID    CPU%    STATE  COMMAND
4120   99.1    R      cpuwork
1884   12.4    S      browser
```

```
...
```

```
Refresh interval: 2.0 sec
```

Important: CPU% conventions differ across tools, especially on multicore systems. Your report must state your normalization convention before comparing values with Linux `top/ps`.

5.3 Controlled CPU Experiment

Use the instructor-provided `cpuwork` program for all required CPU-load experiments so that every student generates a comparable workload. Do not substitute another workload unless the instructor explicitly approves it.

Instructor-provided files: `cpuwork.c` and `Makefile`. Build with `make`, then run `cpuwork` with the required thread counts below.

```
./cpuwork --threads 1
./cpuwork --threads 2
./cpuwork --threads 4
```

For each selected workload, record:

Workload	mytop CPU%	Linux top/ps CPU%	perf task-clock / observation	Interpretation
1 worker/thread				
2 workers/threads				
4 workers/threads				

For each `cpuwork` run, observe the same PID with mytop and at least one standard Linux tool. Use the following `perf` command as part of the required evidence:

```
perf stat -p <PID> -e task-clock,context-switches,cpu-migrations sleep 10
```

5.4 Sampling-Interval Experiment

Choose one CPU-active process and compare a short sampling interval with the required 2-second interval. You may temporarily add a command-line interval option or modify a constant for this experiment.

Interval	Observed CPU% Behavior	Stability / Responsiveness
0.1 sec		
2.0 sec		

5.5 Required Explanation

14. Why can CPU usage not be calculated meaningfully from only one snapshot?
15. What quantities are subtracted between snapshot t1 and snapshot t2?
16. How does your CPU% normalization treat multicore execution? Could one process exceed 100% under another convention?
17. Why may your mytop CPU% differ from Linux top or ps even when both are reasonable?
18. How did changing the sampling interval affect stability and responsiveness of the reported CPU%?
19. How did task-clock and context-switch observations help explain the CPU workload?
20. Which measurement or implementation limitation had the largest effect on your results?

6. Evidence Required for Each Level

Level	Code Evidence	Runtime Evidence	Analysis Evidence
1	Tiny shell + pipeline	strace -f excerpts	fork/exec/wait + FD explanation
2	custom syscall + myps	myps vs Linux ps	task/process/state + user/kernel copy explanation
3	mytop	controlled CPU workload + top/ps/perf comparison	snapshot/CPU%/sampling explanation

Do not submit pages of raw trace output. Select only the lines needed to support your explanation and annotate them with process IDs or short comments.

7. Submission Package

- Source code and Makefile for all student-implemented components. The instructor-provided cpuwork source does not need to be included in your submission unless requested.
- A patch/diff or clearly identified list of kernel files changed for the custom system call.
- README with build, kernel-install/boot, and execution instructions.
- Technical report (recommended 4-6 pages) organized as Level 1 / Level 2 / Level 3.
- Selected strace evidence, myps-vs-ps comparison, mytop performance tables, and perf observations.
- A short AI-use statement: tools used, what they were used for, and at least one generated suggestion/code issue that you verified or corrected, if AI tools were used.

8. Evaluation

Category	Weight	What Is Evaluated
Implementation	40%	Correct shell, custom syscall/myps, and mytop behavior
Experiment & Observation	25%	Appropriate strace/system-tool/perf evidence and controlled experiments
Technical Explanation	25%	Correct connection between evidence and Linux execution concepts
Code Quality & Report	10%	Structure, error handling, reproducibility, clarity

A program that works but is not understood or experimentally validated cannot receive full credit. Conversely, performance values are not graded by absolute magnitude across different machines.

9. Final Check-Off Questions

During grading or demonstration, you may be asked to modify a parameter, reproduce one observation, or explain a selected line of your own code/trace. Examples include:

- Show where the child process reaches `execve()` in your trace.
- Explain what would happen if a pipe write end remains open.
- Explain why your syscall uses `copy_to_user()`.
- Explain one PID/TGID case from your own output.
- Change the mytop sampling interval and predict the effect before running it.
- Explain a difference between your mytop result and Linux top/perf.
- Run cpuwork with a different specified thread count and predict how its CPU usage should change under your documented normalization convention.

Build it. Observe Linux. Validate the result. Explain the evidence.